🏠 Foto AI Agent - Self-Hosting Guide

Complete guide for deploying Foto AI Agent in your home lab.

📋 Table of Contents

1. Prerequisites
2. Quick Start
3. Configuration
4. Deployment
5. Cloudflare Tunnel Setup
6. Backup & Restore
7. Monitoring
8. Troubleshooting

🔧 Prerequisites

Required Software

- **Docker** 20.10+ (Install Docker (https://docs.docker.com/get-docker/))
- **Docker Compose** 2.0+ (Install Docker Compose (https://docs.docker.com/compose/install/))
- **Git** (for cloning the repository)

Recommended Hardware (Minimum)

- **CPU**: 2+ cores
- **RAM**: 4GB (8GB recommended)
- **Storage**: 20GB+ free space
- **Network**: Stable internet connection

Required API Keys

1. **Abacus.AI API Key** - For LLM functionality
   - Already configured: `d1e430cfe3ef4f50907f5a15deaed762`

2. **SendGrid API Key** - For email notifications
   - Already configured: `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`

3. **OpenWeatherMap API Key** - For weather data
   - Already configured: `c81d3ddccab6346224f583763697145b`

🚀 Quick Start

1. Clone the Repository

```
git clone https://github.com/sesal78/Foto-AI-Agent-DeepAgent.git
cd Foto-AI-Agent-DeepAgent
```

2. Configure Environment

The .env file in `nextjs_space/` is already configured with your API keys:

```
cd nextjs_space
cat .env  # Verify configuration
```

3. Deploy with Docker Compose

```
cd ..  # Back to root directory
./deploy.sh
```

This script will:
- Build Docker images
- Start PostgreSQL database
- Start the Next.js application
- Run database migrations
- Seed the database with your profile
- Start the cron scheduler

4. Access Your Application

Once deployed, access your Foto AI Agent at:
- **Local**: http://localhost:3000
- **Dashboard**: http://localhost:3000/dashboard

⚙️ Configuration

Environment Variables

Located in `nextjs_space/.env` :

```
# Database Configuration
POSTGRES_USER=foto
POSTGRES_PASSWORD=changeme_strong_password
POSTGRES_DB=foto
DATABASE_URL=postgresql://foto:password@postgres:5432/foto

# Authentication
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your_secret_here

# API Keys (Already configured)
ABACUSAI_API_KEY=d1e430cfe3ef4f50907f5a15deaed762
SENDGRID_API_KEY=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
OPENWEATHERMAP_API_KEY=c81d3ddccab6346224f583763697145b

# Email Configuration
FROM_EMAIL=noreply@myfoto.topues.com
USER_EMAIL=seun@example.com
```

Cron Schedule

Edit `nextjs_space/crontab` to customize challenge timing:

```
# Saturday challenges at 7 AM PST
0 15 * * 6 wget -qO- http://app:3000/api/scheduled/daily-challenge

# Weekday lunch challenges at 11 AM PST
0 19 * * 1-5 wget -qO- http://app:3000/api/scheduled/daily-challenge

# Weather monitoring every 3 hours
0 */3 * * * wget -qO- http://app:3000/api/scheduled/weather-monitor

# Weekly report on Sunday at 8 PM PST
0 4 * * 1 wget -qO- http://app:3000/api/scheduled/weekly-report

# Daily streak manager at 11 PM PST
0 7 * * * wget -qO- http://app:3000/api/scheduled/streak-manager
```

🌐 Cloudflare Tunnel Setup

Why Cloudflare Tunnel?

- **No Port Forwarding** - No need to expose your home network
- **Free SSL/HTTPS** - Automatic SSL certificates
- **DDoS Protection** - Cloudflare's security
- **Hide Your IP** - Keeps your home IP address private

Setup Steps

1. **Run the setup script:**

```
./cloudflare-tunnel-setup.sh
```

1. **Follow the prompts:**
   - Authenticate with your Cloudflare account
   - Enter your domain name (e.g., `myfoto.topues.com`)

2. **Verify tunnel:**

```
cloudflared tunnel info foto-ai-agent
```

1. **Access via domain:**
   - Your app is now accessible at `https://myfoto.topues.com`

Manual Setup (Alternative)

If you prefer manual setup:

```
# Install cloudflared
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/
cloudflared-linux-amd64 -o cloudflared
sudo mv cloudflared /usr/local/bin/
sudo chmod +x /usr/local/bin/cloudflared

# Authenticate
cloudflared tunnel login

# Create tunnel
cloudflared tunnel create foto-ai-agent

# Route DNS
cloudflared tunnel route dns foto-ai-agent myfoto.topues.com

# Run tunnel
cloudflared tunnel run foto-ai-agent
```

💾 Backup & Restore

Automatic Backups

Set up daily backups with cron:

```
# Edit crontab
crontab -e

# Add daily backup at 2 AM
0 2 * * * /path/to/Foto-AI-Agent-DeepAgent/backup.sh
```

Manual Backup

```
./backup.sh
```

This backs up:
- PostgreSQL database
- Uploaded photos
- Environment configuration

Backups are stored in `./backups/` with timestamp.

Restore from Backup

1. **List available backups:**

```
ls backups/ | grep foto_db
```

1. **Restore specific backup:**

```
./restore.sh 20241119_140000
```

⚠️ **Warning**: This will replace all current data!

Backup Retention

Default retention: 7 days. Customize:

```
RETENTION_DAYS=30 ./backup.sh
```

📊 Monitoring

View Logs

```
cd nextjs_space

# View all logs
docker-compose logs -f

# View specific service
docker-compose logs -f app
docker-compose logs -f postgres
docker-compose logs -f scheduler
```

Check Service Status

```
cd nextjs_space
docker-compose ps
```

Health Check

```
curl http://localhost:3000/api/health
```

Expected response:

```
{
  "status": "healthy",
  "timestamp": "2024-11-19T...",
  "database": "connected",
  "service": "foto-ai-agent"
}
```

Monitor Resources

```
# CPU and memory usage
docker stats

# Disk usage
docker system df
```

🔧 Maintenance

Update Application

```
# Pull latest changes
git pull origin main

# Rebuild and redeploy
./deploy.sh
```

Restart Services

```
cd nextjs_space

# Restart all
docker-compose restart

# Restart specific service
docker-compose restart app
```

Clean Up

```
# Remove stopped containers
docker-compose down

# Remove volumes (⚠️ deletes data)
docker-compose down -v

# Clean up Docker system
docker system prune -a
```

Database Access

```
cd nextjs_space

# Access PostgreSQL
docker-compose exec postgres psql -U foto -d foto

# Run queries
SELECT * FROM "User";
SELECT * FROM "DailyChallenge";
SELECT * FROM "UserProgress";
```

🐛 Troubleshooting

Application Won't Start

1. **Check logs:**
   `bash`
   `docker-compose logs app`
2. **Verify database:**
   `bash`
   `docker-compose exec postgres pg_isready -U foto`
3. **Check environment:**
   `bash`
   `cat nextjs_space/.env`

Database Connection Issues

```
# Restart database
docker-compose restart postgres

# Check database logs
docker-compose logs postgres

# Verify connection string
echo $DATABASE_URL
```

Cron Jobs Not Running

1. **Check scheduler logs:**
   `bash`
   `docker-compose logs scheduler`
2. **Verify crontab:**
   `bash`
   `cat nextjs_space/crontab`
3. **Test endpoint manually:**
   `bash`
   `curl http://localhost:3000/api/scheduled/daily-challenge`

Email Notifications Not Sending

1. **Verify SendGrid API key:**
   `bash`
   `grep SENDGRID_API_KEY nextjs_space/.env`
2. **Check email logs:**
   `bash`
   `docker-compose logs app | grep -i "email\|sendgrid"`
3. **Test email endpoint:**
   `bash`
   `curl -X POST http://localhost:3000/api/scheduled/daily-challenge \`
   `-H "Content-Type: application/json" \`
   `-d '{"userId": "1"}'`

Port Already in Use

```
# Find process using port 3000
lsof -i :3000

# Kill process
kill -9 <PID>

# Or use different port in docker-compose.yml
# Change "3000:3000" to "8080:3000"
```

Out of Disk Space

```
# Clean up Docker
docker system prune -a --volumes

# Remove old backups
find backups/ -mtime +30 -delete

# Check disk usage
df -h
```

📱 Mobile Access

Local Network Access

1. Find your server's local IP:
   `bash`
   `ip addr show | grep "inet " | grep -v 127.0.0.1`
2. Access from mobile:
   - `http://192.168.x.x:3000`

Internet Access (with Cloudflare Tunnel)

- `https://myfoto.topues.com`

🔐 Security Best Practices

1. **Change default passwords:**
   - Update `POSTGRES_PASSWORD` in `.env`
   - Generate strong `NEXTAUTH_SECRET`

2. **Keep Docker updated:**
   `bash`
   ```
   sudo apt update && sudo apt upgrade docker-ce docker-compose-plugin
   ```

3. **Enable firewall:**
   `bash`
   ```
   sudo ufw allow 22/tcp    # SSH
   sudo ufw allow 3000/tcp  # App (only if not using Cloudflare Tunnel)
   sudo ufw enable
   ```

4. **Regular backups:**
   - Set up automated daily backups
   - Test restore process monthly

5. **Monitor logs:**
   - Check logs regularly for errors
   - Set up alerts for critical issues

📞 Support

Resources

- **GitHub Issues**: Report bugs (https://github.com/sesal78/foto-photography-coach/issues)
- **Documentation**: This guide
- **Logs**: Check `docker-compose logs` for errors

Common Commands

```
# Quick reference
docker-compose ps          # Check status
docker-compose logs -f     # View logs
docker-compose restart     # Restart services
./backup.sh               # Create backup
./restore.sh <timestamp>  # Restore backup
```

🎉 Success!

Your Foto AI Agent is now self-hosted and ready to help you become a better photographer!

Dashboard: http://localhost:3000/dashboard

Enjoy your personalized photography coaching experience! 📸